Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3760387

PDF Download
3760387.pdf
26 January 2026
Total Citations: 0
Total Downloads: 351

Published: 01 October 2025
Online AM: 29 August 2025
Accepted: 07 August 2025
Revised: 07 August 2025
Received: 07 August 2025

Citation in BibTeX format

RESEARCH-ARTICLE

Wasm-IO: Enabling Low-Level Device Interaction in WebAssembly for Industry Automation

MAXIMILIAN LEONHARD SEIDLER, University of Erlangen-Nuremberg, Erlangen, Bayern, Germany

ALEXANDER KRAUSE

PETER ULBRICH

Open Access Support provided by:

University of Erlangen-Nuremberg

ACM Transactions on Embedded Computing Systems, Volume 24, Issue 5s (September 2025)
https://doi.org/10.1145/3760387
EISSN: 1558-3465

Wasm-IO: Enabling Low-Level Device Interaction in WebAssembly for Industry Automation

MAXIMILIAN SEIDLER, System Software Group, Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany and Siemens AG, Erlangen, Germany
ALEXANDER KRAUSE, System Software Group, Technische Universität Dortmund, Dortmund, Germany
PETER ULBRICH, System Software Group, Technische Universität Dortmund, Dortmund, Germany

Certification on a component level is highly beneficial in industrial automation because it allows for independent verification and updates without compromising the reliability of the overall system. Containerization technologies naturally address this demand by providing isolation between software modules. In particular, WebAssembly-based (Wasm) containerization is gaining popularity in industrial automation due to its inherent advantages, including cross-platform interoperability and secure execution of untrusted third-party code. However, Wasm's strict sandboxing poses a significant limitation as it severely limits interaction with hardware devices, making it difficult to interface with sensors and actuators. This is a substantial barrier to adoption in industrial automation, where real-time and low-level hardware interactions are critical.

To address this challenge, we present Wasm-IO, a framework designed to facilitate peripheral I/O operations within Wasm containers. Wasm-IO allows the development of isolated device drivers in Wasm, explicitly moving hardware interaction to the container level. Our architectural approach facilitates containers with hardware interaction to be independently certified, updated, and maintained without adversely affecting each other. This article elucidates foundational methodologies and practical implementations supporting synchronous and asynchronous I/O operations and methods for embedding platform-independent peripheral configurations within Wasm binaries. Additionally, we present an extended priority model enabling interrupt handling in Wasm while maintaining temporal isolation. Our evaluation demonstrates that Wasm-IO significantly reduces latency and overhead compared to existing methods and traditional user-level driver implementations, effectively addressing certification and functional requirements critical to industrial automation systems.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; *Real-time system architecture*; • **Software and its engineering** → Software post-development issues;

Additional Key Words and Phrases: WebAssembly (wasm), interrupts, peripherals, certification

Maximilian Seidler also affilliated with Siemens AG..

This work was funded by the German Federal Ministry of Education and Research (BMBF) under grant number 16ME0454 (EMDRIVE). To assist the writing process, Grammarly AI was used.

Authors' Contact Information: Maximilian Seidler, System Software Group, Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany and Siemens AG, Erlangen, Germany; e-mail: maximilian.seidler@fau.de; Alexander Krause, System Software Group, Technische Universität Dortmund, Dortmund, NRW, Germany; e-mail: alexander.krause@cs.tu-dortmund.de; Peter Ulbrich, System Software Group, Technische Universität Dortmund, Dortmund, NRW, Germany; e-mail: peter.ulbrich@tu-dortmund.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1539-9087/2025/09-ART146
https://doi.org/10.1145/3760387

ACM Reference Format:
Maximilian Seidler, Alexander Krause, and Peter Ulbrich. 2025. Wasm-IO: Enabling Low-Level Device Interaction in WebAssembly for Industry Automation. *ACM Trans. Embedd. Comput. Syst.* 24, 5s, Article 146 (September 2025), 26 pages. https://doi.org/10.1145/3760387

1 Introduction

By encapsulating applications in containers, developers can reliably build, test, and deploy software across diverse platforms, given an appropriate runtime environment. Containerization has thus experienced rapid adoption in industrial automation sectors, including manufacturing, rail, and infrastructure, where robustness, security, and reliability are paramount and strictly governed by certification requirements. Moreover, automation systems often have prolonged lifecycles—sometimes spanning decades—during which hardware and software evolve. Therefore, adapting or integrating new components without compromising system integrity or certification is crucial.

Developing industrial automation systems is typically a collaborative process between an Original Equipment Manufacturer (OEM) and its customers (i. e., operators). OEMs supply hardware components and execution platforms, such as Programmable Logic Controllers (PLCs), runtime environments, and base applications. Operators subsequently extend these platforms by integrating additional application components tailored to their operational requirements. Consequently, OEMs and operators must manage evolving feature sets, dynamic operational demands, and stringent certifications throughout extensive system lifecycles, frequently in regulated environments under substantial cost pressures.

To illustrate the practical challenges, let's look at a Train Control System (TCS). These systems rely extensively on sensors, actuators, and onboard computing platforms to monitor and control critical aspects such as train speed, position, and track conditions. Due to their safety-critical nature, TCSs must adhere to stringent regulatory standards and certification requirements. Additionally, these systems typically have prolonged service lives and must remain operationally robust throughout their entire lifecycle. A single train design may be produced across many years or even decades, and the same control software functionality could be integrated into different train types or configurations. Consequently, manufacturers and models of installed sensors and actuators may vary due to availability, cost considerations, or technological advancements. Parallel to these hardware variations, regulatory bodies frequently mandate updates or the introduction of new capabilities. Such regulatory-driven adaptations can require updating or adding peripheral components to existing TCS installations. Moreover, operators often wish to incorporate improvements from the rapidly evolving field of software algorithms, e. g., artificial intelligence for predictive maintenance or advanced diagnostic capabilities. Each of these developments imposes significant demands on the flexibility and modularity of the underlying software and hardware architecture.

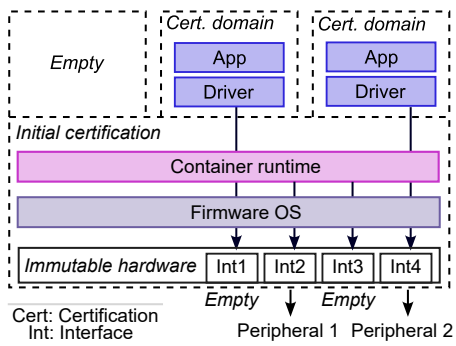


Fig. 1. A Wasm-based architecture enabling independent certification domains by facilitating execution of containers with strong isolation and peripheral interaction capabilities

WebAssembly (Wasm) has recently emerged as an alternative approach to traditional containerization, including embedded and safety-critical domains, promoting fine-grained certifiability. Wasm's inherent memory safety offers robust isolation and abstraction. Its platform-agnostic and

compact design ensures portability across various hardware and software environments—a critical requirement for operators managing diverse system deployments. Additionally, Wasm streamlines software, allowing new binaries to be loaded without reflashing firmware or device reboots. Thus, Wasm is increasingly recognized as a promising technology addressing critical constraints of containerization in industrial and embedded contexts, facilitating scalable, efficient, secure, and certifiable applications [27, 28, 32, 34, 37, 44, 50, 56, 60].

Consequently, the industry envisions a Wasm-based containerization architecture, as shown in Figure 1, where an initial certification covers all OEM-provided components, including base firmware, and the Wasm container runtime. This initial certification would be performed once, clearly defining the OEM's liability boundary. Operators could then independently modify or add application containers within their certification domains and peripherals by leveraging hardware interfaces, simplifying the (re-)certification process and enabling greater operational flexibility.

1.1 Problem Statement

Adopting such a Wasm containerization architecture within safety-critical and real-time contexts remains in its infancy. A significant challenge arises from Wasm's strict isolation mechanisms, which restrict direct (i. e., low level) interaction with hardware peripherals within containers. Typically, such hardware interaction is conducted through Memory Mapped I/O (MMIO), whereby drivers directly access device registers. However, this process is inherently memory-unsafe and platform-specific. Even in type-safe languages, MMIO requires interpreting numeric addresses as references, which conflicts fundamentally with Wasm's portability and safety principles.

Thus, while Wasm is highly advantageous for modular application containerization, its inability to naturally handle hardware-dependent software constitutes a critical barrier. Consequently, enabling device drivers within Wasm remains an essential challenge for effective Wasm adoption in industrial automation systems—which we address in this article. Our primary objective is to add synchronous and asynchronous I/O support to Wasm while maintaining its isolation properties and platform independence. Along the way, we have identified the following challenges:

(1) Architectural Challenge—*Designing Device Drivers and Interrupt Handling.* To preserve Wasm's portability and avoid recertification due to modifications of the base runtime or platform, we must ensure that drivers reliably link to the platform's available and assignable interfaces (e. g., serial, bus, analog) that remain unchanged. This requires mapping memory accesses to hardware registers without prior knowledge of platform specifics at deployment time. Furthermore, many drivers mandate asynchronous handling of hardware interrupts with specific real-time capabilities. To this end, Wasm's isolation model neither supports low-latency interrupt handling nor is suited to integrate with the priority levels of other tasks and interrupt handlers.

Our Approach: Wasm-IO facilitates the development of platform-agnostic device drivers in Wasm, supporting both synchronous and asynchronous I/O with minimal interference. Our solution leverages the established concept of devicetrees, using a new binding to identify and generate unique names for device properties (e. g., register addresses, interrupts). We embed these labels in Wasm binaries, enabling matching and verification at load time without prior knowledge of hardware specifics and ensuring coherence and portability. For interrupt handlers, we introduce an extended priority model incorporating a conventional Operating System (OS) two-stage handling approach, adding a Wasm-specific deferred handler level. This structure enables the sequentialization of Wasm handlers while preserving temporal isolation and mitigating interference. With its methodology, Wasm-IO takes a major step toward containerization architectures as shown in Figure 1.

(2) Runtime Challenge—*Secure Memory Access.* Wasm's strict isolation mechanisms prevent native peripheral interactions. Thus, the runtime must provide secure, tightly controlled mechanisms

enabling peripheral access by Wasm modules while ensuring comprehensive host control and maintaining runtime isolation.

Our Approach: Wasm-IO implements two alternative approaches for interacting with the low-level periphery. Developers can use an API to access hardware registers by labels or have the hardware registers be mapped to the container's address space by our Wasm-IO runtime. In the latter, store and load accesses to the mapped addresses are automatically intercepted. Both methods check accesses to actual hardware registers based on permissions.

***(3) Cost Challenge**—Overheads and Predictability.* Wasm-IO's strong conceptual isolation capabilities are one aspect of its feasibility. However, its practical impact in real-world scenarios is tied to the induced overheads, latencies, and, even more importantly, timing predictability. In particular, we must quantify the overheads associated with synchronous I/O and establish upper bounds for execution paths.

Our Approach: To mitigate overheads and hide latencies, we developed multiple I/O access variants, executed at the kernel level and in userspace, and conducted measurement-based timing analysis to characterize them. We also identified worst-case paths, which allow estimating Worst-Case Execution Times (WCETs) for a given deployment platform [31]. Our characterization demonstrates the potential of Wasm-IO in safety-critical and real-time applications.

1.2 Contributions and Outline

With Wasm-IO, we introduce a novel approach for integrating device drivers and interrupt handlers into Wasm, specifically addressing the challenges in industry automation highlighted previously. We present relevant background in Section 2. Subsequently, we make three significant contributions:

(1) Wasm-IO, a methodology to systematically introduce low-level driver and interrupt support to WebAssembly, preserving strong spatial and temporal isolation, enabling containerization, and facilitating design-time analysis and partitioning is presented in Section 3.
(2) Our runtime provides tailored memory checks that enable secure peripheral access, effectively hiding access latencies and ensuring predictable interrupt handling. We detail Wasm-IO's prototype in Zephyr RTOS in Section 4.
(3) We provide a thorough, measurement-based analysis of overheads, Worst-Observed Execution Times (WOETs), latencies, and worst-case paths for each implementation variant in Section 5, substantiating the applicability of our approach in industrial real-time applications.

We address related work in Section 6. Section 7 concludes our work.

2 Background

This section provides background information on certification, OS specifics, devicetrees, and Wasm and its isolation concepts relevant to embedded and industrial systems.

2.1 Certification in Industry

Operators utilize PLCs to implement industrial control systems. Since they interact with the physical world, failures may lead to significant physical consequences, such as production loss, equipment damage, or even life-threatening situations. Consequently, minimizing hazardous failures is a critical requirement in industry. To address this need, several standards, including IEC 61508 [4], have been established. This standard defines Safety Integrity Levels (SILs) ranging from 1 to 4, with SIL 1 indicating the lowest threat and SIL 4 the highest in cases of failure. Part 3 of the standard outlines software requirements, emphasizing guidelines for the software life cycle and the prevention of systematic failures. While IEC 61508 generally addresses programmable electronic systems, EN 50128 [5] provides refinements for railway systems.

According to IEC 61508, software systems should prevent failures, including those stemming from timing issues, (unintended) misuse, and human errors. The standard recommends that systems be structured into manageable modules, mitigating interference and error propagation while clearly expressing timing constraints and sequencing. Moreover, dynamic loadable elements should be avoided unless their integrity can be verified using offline tools.

Certification entails demonstrating compliance with established standards, either through self-assessment or a third-party audit. This process requires substantiation of adherence to recommended development practices, proper documentation, and verification of functional correctness, robustness, and safety through testing and static analysis of all relevant components, including basic software components and tools. An equivalent process applies in case of modifications, necessitating re-verification. In the context of certification to IEC 61508, the extent and rigor of (re-)verification are contingent upon the assigned SIL level [4, 45].

To empower operators to comply with the standard, OEMs must provide runtime frameworks that strictly adhere to the standard's requirements, specifically preventing malicious access and ensuring temporal isolation. Wasm-IO provides the means to advance OEMs toward such a platform, particularly concerning modifications. IEC 61508 mandates the re-verification of modified modules for SIL 1, all affected modules for SIL 2, and the entire system for ≥SIL 3, with static tools analysis recommended for ≥SIL 3. The proposed framework confines the impact of modifications to only the changed modules, thus rendering the SIL 2 and SIL 1 requirements effectively equivalent. Moreover, the isolation strategy minimizes interference and side effects, thereby streamlining the inputs necessary for static verification tools, improving error tolerance, and assisting ≥SIL 3 certification.

2.2 Traditional Interrupt Model

To mitigate timing interference of handler execution and system unresponsiveness due to interrupts being disabled, OS-level interrupt handling and synchronization typically follow a strict priority model [42] with three priority levels: E_i $i \in \{0, 1/2, 1\}$. A level E_i obeys the following rules:

(1) It can be interrupted at any time by E_j for $j > i$.
(2) It is not interrupted by control flow on E_k for $k \leq i$.
(3) Control flows on E_i run sequentially (*run to completion*).

As illustrated in Figure 2, the level E_0 is utilized for the application code. In contrast, any interrupt runs on E_1. This, often called *prologue* handles the immediate hardware response and time-critical operations (e. g., copying data from the device). A prologue may issue an *epilogue* on $E_{1/2}$, which performs further processing that can tolerate delay (e. g., aggregation or activating application tasks) with interrupts enabled.

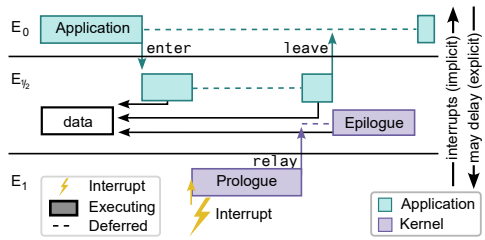


Fig. 2. An example prologue-epilogue interrupt synchronization according to the priority level model.

Data shared between interrupt handlers and application tasks conceptionally resides at priority level $E_{1/2}$. Due to sequentialization rules, no additional synchronization is needed when executed on a single-core system [42]. Switching between levels is done explicitly via three system calls: *enter* and *leave* switch between $E_{1/2}$ and E_0, while *relay* requests an epilogue from E_1. In Linux the same concept is referred to as *top half* (prologue) and *bottom half* (epilogue) [33]. Wasm-IO adopts and expands this model conceptually.

2.3 Devicetrees

The *devicetree* specification [7] addresses the need for a common understanding of hardware interfaces in computer systems, particularly embedded devices. It defines a hierarchical tree structure where each node represents a device or component and contains properties describing its characteristics, including register addresses and configuration. Devicetrees can also represent interrupt sources, controllers, and numbers as node properties. Nodes can be identified by their *labels* or hierarchical paths. While not explicitly stated, the hierarchical structure naturally aligns with hardware levels: CPU, System On a Chip (SOC), and peripherals. The first two are considered immutable due to requiring board modifications, while the peripheral level allows for flexible replacement of sensors, actuators, and other external devices. In Zephyr, the basis for our prototype, the devicetree serves exclusively for software design and compilation during the development phase.

2.4 WebAssembly Fundamentals

WebAssembly (Wasm) [6, 21] was introduced as a performance-focused yet sandboxed alternative to JavaScript. Designed as a low-level, formally defined binary instruction format, Wasm emphasizes strong isolation yet bolsters support for various source languages. Its bytecode is portable across diverse hardware platforms due to its abstract instruction set. WebAssembly is tied to a dedicated execution environment and can be compiled Ahead Of Time (AOT) or Just In Time (JIT) to native instructions or directly executed via interpretation. This flexibility has led to Wasm's adoption in fields beyond web development, including embedded and edge computing. However, Wasm's indirection inherently introduces overhead, which varies based on runtime optimization (such as JIT or AOT compilation) and the specific implementation. In the literature [32, 36, 37, 50, 51, 56, 60], the overhead factors on embedded devices range from 1.14 to 36.15 in comparison to native performance, with interpreters generally exceeding a factor of 10.

As a stack-based virtual machine, Wasm instructions manipulate an operand stack, allowing values to be popped, manipulated, and pushed. Per the Wasm 1.0 specification [6] and its WebAssembly Micro Runtime (WAMR) implementation [8, 9], a Wasm executable consists of a *module* defining static components, such as bytecode, function definitions, tables, and global values. When instantiated, a *module instance* and an *execution environment* are created. The latter then maintains a call stack, operand stack, and references to the module instance and the current function. WebAssembly provides robust spatial isolation through its memory model and type system. Wasm bytecode can only access its own dedicated linear memory, i. e., its heap, through 32-bit[1] integer indices. Type consistency is enforced through static checks. The type system prevents out-of-bounds accesses at the lower limit, while the execution environment ensures the upper memory limit. Accessing the host system's memory is not feasible. Pointers are represented as 32-bit offsets in global tables, such as the function table, eliminating dereferenceable types and, thus, the risk of malicious memory access. Control flow is protected by keeping the call stack separate from the operand stack, with control instructions constrained to prevent stack-based attacks and invalid jumps [21].

Nonetheless, Wasm provides mechanisms to interact with the host system by *importing* functions. Imported functions transfer control from Wasm bytecode to native code, potentially breaking the isolation. The WebAssembly System Interface (WASI) [20] defines system interfaces, offering file system access, and other low-level interactions for non-web environments.

Wasm's formal specification supports the verification of the bytecode design and type system correctness [52, 53]. Deducing interpreters from the formal rules facilitates the development of formally-verified interpreters. While formally generated interpreters often exhibit slower execution times, Watt et al. [54] indicate that their interpreter performs comparably to the debug builds of

[1] 64-bit memory indices are subject of an official Wasm standard modification proposal [17] approaching standardization.

industry-standard interpreters. Although formally proven interpreters for resource-constrained devices are currently lacking, we expect further research to close this gap driven by the increasing adoption of Wasm. Meanwhile, verification remains an open question for AOT and JIT compiling [60]. Consequently, this article builds upon interpreters and assumes, where relevant, a correct runtime. The correctness of the runtime is achieved either through verification or adequate testing.

3 Wasm-IO: Design and Concept

The scenario presented possesses several unique requirements for PLC software. The objective is to provide a container runtime that facilitates adding or modifying business logic and peripherals, including their drivers. However, spatial or temporal influences may invalidate the initial firmware certification, e. g., through memory manipulation or execution delays. Moreover, faults must not propagate and entail unrecoverable states. Updates must seamlessly be integrated and run across diverse PLC hardware platforms. To meet these requirements, we developed Wasm-IO, with the following section providing an in-depth analysis of the underlying concepts and design principles.

3.1 System Model

Wasm-IO enables low-level peripheral interaction within Wasm environments. This article focuses on containerization architectures, as illustrated in Figure 1, deployed in OEM-supplied industrial devices. In such an architecture, the base system—including hardware, interfaces, firmware, and the Wasm runtime—is statically verified and certified, remaining unchanged throughout the device's operational lifetime. Applications implemented as Wasm containers can thus be developed, certified, and maintained independently.

Application containers may use predefined I/O interfaces provided for integration. Each interface is exclusively assigned to a single container, ensuring deterministic behavior. Shared peripheral access requires dedicated proxy containers, typical in microkernel architectures, with deterministic communication provided by OEM firmware. Firmware-level drivers cannot be replaced by containers, maintaining a strict boundary between certified OEM-provided components and containers.

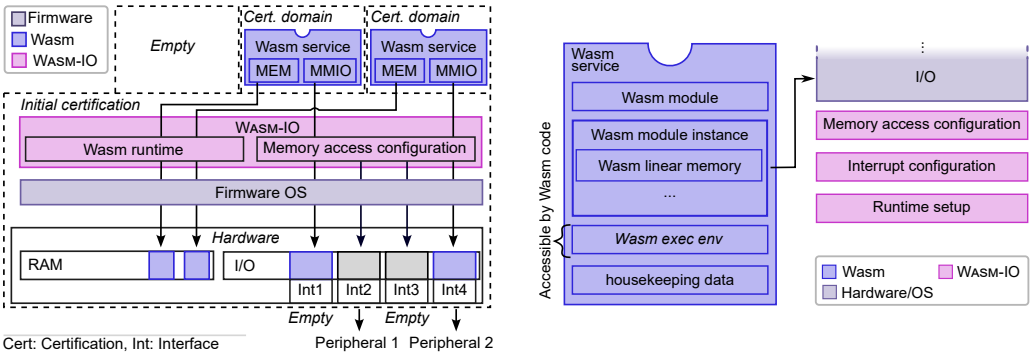
For temporal isolation, we assume a hierarchical scheduling with predefined timing budgets to account for the temporal interference introduced by Wasm-IO drivers. Each container and associated Wasm-IO interaction is assigned a specific time budget to maintain predictability and isolation. Scheduling specifics are beyond this article's scope, but their correctness is assumed.

3.2 Architecture

Subsequently, we detail the architectural aspects of Wasm-IO and a refined version of the containerization architecture as illustrated in Figure 3(a). Here, Wasm-IO acts as an intermediate kernel-level layer to execute application containers in isolation, incorporating a Wasm runtime to leverage Wasm's sandboxing. For spatial isolation, a formally verified Wasm runtime must be used (see Section 2.4). Temporal isolation and I/O capabilities are managed by Wasm-IO.

To modify the PLC's capabilities, operators may load containers providing services that are not part of the firmware. These containerized services, the *Wasm services*, may include business logic, driver logic, or both. Each Wasm service is single-threaded, isolated from other services and the OS, and executed within an OS thread running the Wasm runtime. The runtime provides isolated RAM segments, while Wasm-IO assigns exclusive interfaces for integration of peripherals.

The relevant parts of Wasm-IO are depicted in Figure 3(b). It provides a *runtime setup* to manage the initialization process of the Wasm runtime. Each service consists of the *Wasm module*, i. e., the binary; the *Wasm module instance*, i. e., global storage; the *Wasm execution environment*, i. e., the state of the stack machine; and some housekeeping data.



(a) (b)

Fig. 3. (a) Wasm-IO's architecture with Wasm services. Wasm-IO ensures service isolation while offering synchronous and asynchronous I/O capabilities. (b) Wasm-IO's memory architecture detailed. Wasm services comprise of Wasm intrinsics and housekeeping data.

The *memory access configuration* includes mapping device registers to a Wasm service, enabling synchronous I/O. The *interrupt configuration* enables asynchronous interaction. The operator's Wasm services are restricted to memory regions within the execution environment (see Section 2), thus unable to modify the access configuration.

3.3 Peripheral Identification and Access

As presented by Challenge 1, changing code that accesses the hardware requires recertifying the entire system. This dilemma could be resolved by providing all existing I/Os with certified interfaces. However, Wasm services then must reference *specific* interfaces provided by *specific* platforms, which fundamentally undermines the aim of cross-platform implementation and jeopardizes the desired independent certification. Essentially, we face the following sub-problems: (a) ensuring the availability of registers and interfaces required by the Wasm service on the device, (b) embedding them in a platform-agnostic manner, and finally, (c) enabling usage within the Wasm service according to the driver's assumed layout while maintaining spatial isolation.

ad (a): Ensuring Availability. Wasm-IO obeys design-time configuration by the OEM to allow an interface to be used within a Wasm service to maintain predefined certification domains. Accordingly, interface registers are labeled and, with their physical addresses, written in the memory access configuration (see Figure 3(b)) during firmware development. We denote these as *exposed registers*. Our implementation (Section 4) shows how this naturally aligns with the concept of devicetrees and our novel devicetree binding. Upon loading a Wasm service, Wasm-IO will verify that all requested registers are exposed. If any dependencies are unavailable, the Wasm service will be rejected. Thus, Wasm-IO prevents any potential conflicts at load time.

ad (b), (c): Expressing Peripheral Requirements and Usage. A driver for a peripheral device is typically programmed against a Hardware Abstraction Layer (HAL), providing placeholder bindings (e. g., macro definitions in C) for each register. The bindings are resolved at compile time employing a board-specific configuration. Consequently, the resulting binary only contains physical addresses. Wasm-IO builds upon this established approach. Rather than resolving bindings at compile time, Wasm-IO introduces a dummy register for each placeholder. These dummy registers connect the compile time binding to a unique register label—instead of a physical address—that remains within the retained Wasm binary when compiling a Wasm service. As detailed in our implementation

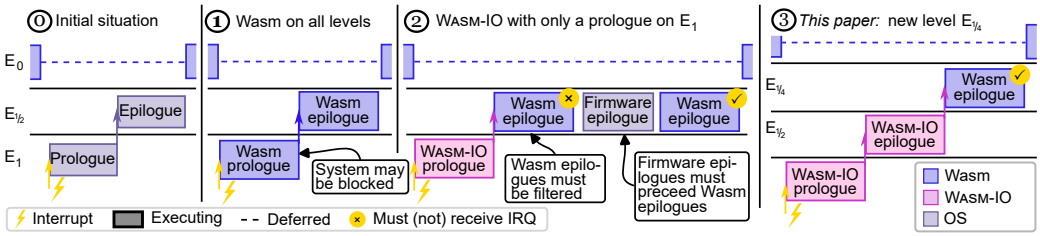


Fig. 4. The deduction of the new extended prologue-epilogue model. Step ⓪ shows the initial solution, step ① blocks system responses. Step ② lacks a trusted entity to ensure proper filtering of Wasm epilogues and precedence of firmware epilogues. Step ③ presents the final solution.

(see Section 4), we utilize global variables for this purpose. Upon loading the service, the memory access configuration enables resolving each dummy register to a physical address via its unique label. Thereby, the dummy registers enable deferral of address resolution until load time, while preserving compile-time information. WASM-IO offers a build tool that streamlines this process. Further, WASM-IO's various methods for manipulating register values limit access to only those addresses with explicit permission.

3.4 Interrupt Handling

Communication from a device to its driver typically occurs asynchronously via interrupts. Contemporary OSes (see Section 2.2) offer a two-stage interrupt handling: the prologue facilitates communication to the device, i. e., data reception, and the epilogue performs the data processing. This initial situation is illustrated in Figure 4 as step ⓪. The Wasm service operates on level E_0, allowing it to be deferred, while any prologue or epilogue is executed on level E_1 or $E_{1/2}$, respectively.

Step ① illustrates the prologue-epilogue model being applied naively. As a Wasm service executes arbitrary code, allowing it to run its own prologue and epilogue introduces significant isolation vulnerabilities. Untrusted code is executed in the prologue with the highest priority, and disabled interrupts. Consequently, any program failure inevitably results in a non-recoverable device failure. Moreover, system determinism is invalidated due to the uncertain execution time and termination. Therefore, WASM-IO offers a general-purpose *WASM-IO prologue* that retrieves data from the device on behalf of the Wasm service. As shown in step ②, this eliminates the vulnerability inherent in the prologue. The WASM-IO prologue controlling the copy process is designed to exhibit bounded execution times, allowing it to be part of the certified firmware and running on E_1.

In step ② firmware and Wasm epilogues are executed at level $E_{1/2}$. However, the execution time of Wasm epilogues remains indeterminate during firmware design and certification. To ensure system determinism, the execution of Wasm epilogues must be handled inferiorly relative to firmware epilogues. Accordingly, a new priority level, $E_{1/4}$, is established, specifically for Wasm epilogues.

Moreover, to uphold the isolation guarantees of WASM-IO, an additional trusted instance is required prior to scheduling a Wasm epilogue. This instance must enforce permission rules for interrupts, e. g., when devices share interrupt lines, and deterministic ordering of Wasm epilogues and prevent monopolization of $E_{1/4}$. Finally, it must prepare the Wasm runtime for the execution as an interrupt routine. However, the prologue must remain concise, while the Wasm epilogues are untrusted. Therefore, we introduce the *WASM-IO epilogue*, a trusted and certified entity addressing these responsibilities. The resulting prologue-epilogue model of WASM-IO is depicted in step ③. WASM-IO provides enter and leave functions that allow Wasm applications to temporarily elevate their priority to $E_{1/4}$, enabling synchronization with their Wasm epilogue. On incoming interrupts, the WASM-IO epilogue may reduce priorities of entered Wasm applications or corresponding

Wasm epilogues to prevent monopolization of $E_{1/4}$ by a malicious Wasm service exploiting the enter call.

Wasm-IO's interrupt synchronization model ensures that firmware prologues (E_1) and epilogues ($E_{1/2}$) consistently take precedence over Wasm epilogues. Both the Wasm-IO prologue and epilogue have with bounded execution times. OEMs account for their schedulability in system design.

To enable a Wasm service to receive data from a device, we facilitate to register data to be received alongside the corresponding interrupt. Prior to scheduling a specific service's epilogue, the Wasm-IO epilogue copies data to the Wasm linear memory with the dummy registers modified accordingly. As the Wasm service then operates on this *copy*, the necessity for the Wasm epilogue to enter any level below $E_{1/4}$ to retrieve the data is eliminated. The primary objective of this method is to significantly reduce interrupt latencies, particularly with userspace enabled.

Additionally, with our revised data model, this approach facilitates timely and deterministic interrupt synchronization, thereby effectively addressing the second part of Challenge 1. Wasm-IO takes responsibility for data reading (Wasm-IO prologue), movement across the different priority levels, and availability in the Wasm epilogue.

4 Implementation

We built our prototype of Wasm-IO based on Zephyr RTOS and the classic WAMR interpreter as interpreters can generally be verified, and no such compilers are on the horizon. Additionally, due to limitations in RAM, we adjusted the page size of Wasm.

4.1 Host-Side Peripheral Identification

We utilize devicetrees as a well-known and suitable data structure to create the memory access configuration. During firmware design, OEMs must identify all interface nodes, e. g., buses or GPIOs, where peripherals can connect. Each of these nodes is referenced in a node of our newly introduced binding type. Our build tool then generates the build-time part of the memory access configuration, producing a list of structures mapping unique names derived from the node's label to properties like interrupt numbers and register addresses. The naming convention is flexible, provided that names remain unique and reproducible, which is achieved through the consistent use of the same name-generating mechanism for host and Wasm service.

Figure 6 illustrates memory access configurations based on access types elaborated later. Name generation uses the devicetree node's label (e. g., `spi1` or `gpioa`) combined with specific registers (e. g., `mode` or `pupd`), if needed, shown on the left side of each configuration in Figure 6. Each entry also includes a reference to the Wasm service, an optional mask, and the host-side element for I/O operations, i. e., the physical register address or a driver reference. The primary aim of the memory access configuration is to associate a unique, device-independent name with this device-specific element for I/O, thereby facilitating resolution at load time. Notably, while device-independent, the naming of each entry reflects its intended function. For example, `mode` and `pupd` registers are architecture-specific; however, these entries are only required when developing GPIO drivers, which inherently require access to their respective registers.

4.2 Service-Side Peripheral Identification

To port existing drivers for Wasm-IO, developers enrich a board-specific HAL with our Wasm-IO bindings. In our prototype, these bindings (e. g., C macro definitions) map to global variables rather than constant register addresses or interrupt lines, as shown in Figure 5. Furthermore, the Wasm-IO bindings file introduces an initialization routine (`_wasm_io_init()`). Upon loading, our runtime invokes this routine, which requests the variable's value, i. e., the physical register address, by querying it via its devicetree label (`get_io(<name>)`). This process is depicted on

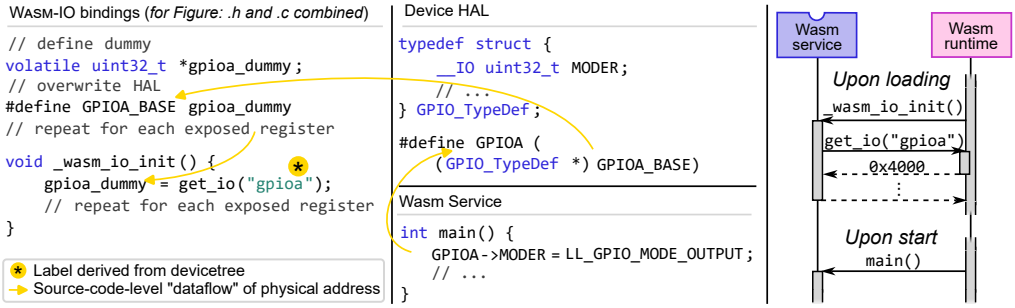


Fig. 5. Resolution process of dummy registers: The Wasm-IO bindings integrate into the device HAL, substituting addresses with global variables. These variables utilize devicetree labels to request physical addresses during loding.

the right of Figure 5. The runtime traverses the memory access configuration established during firmware compilation, yielding the corresponding physical address. This mechanism effectively links placeholder variables via the unique devicetree labels to the physical addresses from the initial firmware build time.

The placeholder variables seamlessly integrate into HAL-based development, as illustrated by the yellow arrows in Figure 5. As these variables are the registers' physical addresses from the Wasm service perspective, we referred to them as dummy registers in Section 3. The devicetree labels are embedded into the custom section of the Wasm binary for load-time verification against the memory access configuration, ensuring the availability of all necessary dependencies.

While devicetrees are a convenient data structure for representing hardware and consistently generating names, our approach is not strictly reliant on it. Consequently, porting our approach to different RTOSes requires minimal effort. To accommodate other source languages, a mechanism must be established to convey names and retrieve their corresponding device-specific values at load time. Our runtime implementation leverages the `_wasm_io_init()` routine, allowing any Wasm binary that provides this function to be applicable, regardless of its source language.

When compiling a source program to Wasm, structures, e. g., for driver registers, are maintained if constant propagation is prevented. We achieve this by using global variables for the dummy registers, which still provide limited optimization capabilities.

Generally, Wasm lacks a mechanism to represent volatile variables and memory consistency principles, concepts commonly present in C/C++-based drivers. However, interpreters directly utilize the Wasm bytecode generated from the C/C++ source, where memory access order remains known. For subsequent compilation steps, such as JIT and AOT compilation, the information on memory order is lost but can be emulated through the use of atomics [22], which have yet to be standardized.[2]

4.3 Synchronous Peripheral Access

In Challenge 2, we indicate the absence of peripheral interaction in Wasm. With Wasm-IO, we provide one reference and four novel methods for synchronous peripheral access: OS-API, R-API, MMIO, and R-API and MMIO with Direct Memory Access (DMA) acceleration.

Operating System's API (OS-API). The OS-API is based on existing literature [25, 29, 44] and serves as a baseline. It enables a Wasm service to access a (certified) OS peripheral driver API. Therefore, a Wasm service must be programmed using the OS driver's APIs. To facilitate platform-agnostic

[2]Atomics is a proposed change to the Wasm specification, arising from the threads proposal [55] in phase four.

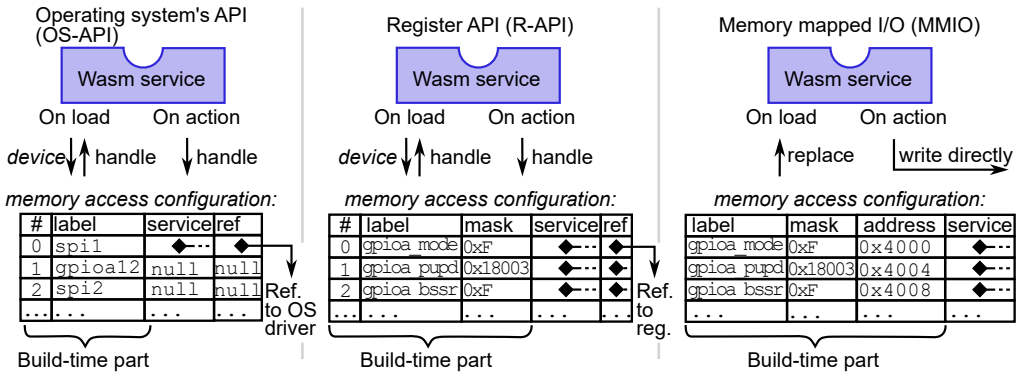


Fig. 6. Wasm-IO's different methods of accessing peripherals: the Operating System's API (OS-API) uses the OS's device driver, the register API (R-API) allows manipulation of registers via an API, and the MMIO approach directly modifies MMIO regions. The memory access configuration links permissions defined at build time when loading Wasm services.

identification of drivers, we incorporate the devicetree node's name of the instance (e. g., `spi1`) into the Wasm binary, instead of registers (e. g., `spi_sck`). Access to the API is facilitated through Wasm-IO, which directly mirrors the OS driver's interface. At load time, the Wasm service requests a handle reference for the desired driver instance instead of resolving dummy registers. This method is illustrated on the left of Figure 6. If the request is approved, the handle is stored in the memory access configuration.

At runtime, the Wasm service calls the OS-internal API for each I/O action via this handle, e. g., `spi_write(...)`. Wasm-IO checks whether the service has the necessary privileges to the handle by reviewing the memory access configuration. If access is granted, the request is forwarded to the corresponding driver. Since this approach relies entirely on the OS-specific driver and its configuration, the OS-API does not permit arbitrary peripheral access, which is the primary objective of this work. Therefore, it is intended solely as a reference.

Register API (R-API). To facilitate access to *arbitrary* peripherals, we introduce the novel R-API. It refines the presented concept of an API, allowing for direct manipulation of registers. Unlike the OS-API, where each handle corresponds to an OS driver, in R-API, each handle refers to a specific register, shown in the center of Figure 6. As a result, a service can interact with any device as long as the corresponding register addresses are known and exposed (see Section 4.1). Consequently, interacting with a register involves calling a function instead of dereferencing a pointer, like in C-like languages. Accordingly, Wasm-IO offers functions such as `get_bit(<handle>)` and `read_register(<handle>)` along with equivalent write operations. However, this design enables Wasm-IO to verify every register access by consolidating the memory access configuration. In addition, this allows utilization of programming languages without a reference concept. For precise control, Wasm-IO applies masks to the values prior to access, e. g., when manipulating GPIO registers, as they provide access to multiple I/O pins. These masks are obtained from the devicetree during the firmware build process and incorporated into the build-time part of the memory access configuration. Registers are sorted by their usage frequency. In one of our specific evaluation cases where Wasm-IO operates within userspace (uncertified scenario), the API call is equivalent to a system call.

Memory Mapped I/O (MMIO). As demonstrated in our evaluation (see Section 5), both the OS-API and the R-API exhibit considerable latency. Therefore, we designed a significantly faster variant enabling true Memory Mapped I/O in Wasm. Instead of exposing host functions as an API, we

modified the bounds check of store and load operations performed by the Wasm runtime. Since I/O register addresses (casted to unsigned integers) are located beyond the linear memory, this check will inevitably result in a fault. If the bounds check fails, Wasm-IO determines whether the address corresponds to an exposed register address and, if it does, reauthorizes the respective instruction. This approach is illustrated on the right of Figure 6. The MMIO method similarly uses masks to restrict access to specific bits within a register. When operating in userspace, Wasm-IO utilizes a system call to read from or write to the register address.

Hardware Acceleration. As demonstrated in our evaluation in Section 5, the primary cause of increased I/O latency in any solution is context switches ([user ⇆] Wasm ⇆ kernel). For situations with high data rates, e. g., sensors with high sampling rates, or data split across multiple registers, this may lead to significant performance impact. To address this issue, we utilize the DMA controller. Additionally, we introduce the concept of the *conveyor memory*, as shown in Figure 7. The Wasm runtime allocates additional memory, which follows the linear memory of the Wasm module. We modify the default Wasm memory bounds check to test against a shifted boundary, including the Wasm linear memory and the conveyor memory. As a result, for the Wasm service, access to Wasm linear memory—i. e., the service's regular program memory—is effectively equivalent to accessing the conveyor memory.

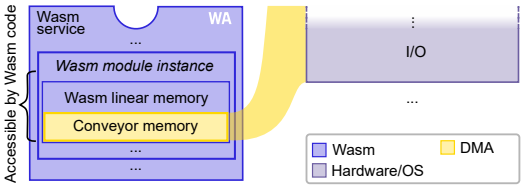


Fig. 7. DMA acceleration: the conveyor memory precedes the Wasm linear memory obviating access checks and enabling DMA-based synchronization with I/O.

At load time, during the resolution of dummy registers, Wasm-IO configures the register addresses in the DMA controller instead of substituting the global variables. Instead, the global variables are initialized to addresses within the conveyor memory. As a result, interactions with registers render a Wasm service to read and write within the conveyor memory. Therefore, any memory access made by the service remains within its accessible memory area, obviating additional access checks by Wasm-IO. During Wasm service execution, the DMA controller continuously polls the registers and conveyor memory, promptly updating any changes. Operating in privileged mode, the DMA controller can access both memory regions, MMIO in the kernel and Wasm, potentially in userspace. Furthermore, the inherent overhead of the Wasm interpretation with multiple cycles per Wasm instruction hides memory latencies, ensuring the timely arrival of updates.

In our prototype, the DMA controller utilizes two channels per register, allowing for bidirectional data transfer between the register and the conveyor memory. It is triggered by a timer interrupt for cyclic polling, alternating both directions. To permit immediate handling of consecutive read/write operations, the timer frequency must significantly exceed the Wasm execution frequency, which is consistently met since interpreters typically exhibit a factor of 10 or more compared to native performance. For multiple consecutive registers, DMA controllers typically support incrementing the source and target addresses. Importantly, services are infeasible to access the DMA configuration directly. This configuration is managed exclusively by Wasm-IO, effectively mitigating DMA attacks that exploit the DMA controller for unauthorized memory access.

The DMA approach is solely designed for acceleration and, thus, does not provide any API. We implement this acceleration transparently in the runtime for both R-API and MMIO, significantly alleviating CPU load and reducing I/O latency. However, allocating two DMA channels per register may be excessive, and continuous polling exerts considerable pressure on the DMA bus. Hybrid solutions with DMA acceleration solely for high-frequency access to selected registers, or performing

DMA transfers only during reads or writes, may effectively balance performance and system impact. Additionally, temporal analysis necessitates an additional hardware model and either careful DMA scheduling or a second DMA controller, as seen in our evaluation platform.

4.4 Interrupt Handling

Wasm-IO offers a platform-independent method to handle interrupts: As with exposed registers, a Wasm service registers as an interrupt handler by specifying an interrupt label along with an optional priority level. The Wasm service provides the index into the Wasm function table via the Wasm-IO-provided API, included in the bindings header. Additionally, the data source from which to copy interrupt data must be defined when employing the data copying mechanism.

Upon the reception of an interrupt, the Wasm-IO prologue sequences the Interrupt Requests (IRQs) by adding them to the epilogue queue, buffers data if registered, and clears the interrupt flag on behalf of the Wasm service. Reset values are derived from devicetree properties.

The Wasm-IO epilogue OS thread awaits items in the epilogue queue. If multiple peripherals share a single interrupt line, it identifies the relevant Wasm epilogues associated with a particular peripheral. De-multiplexing is performed utilizing the memory access configuration. Subsequently, the data is copied to the Wasm epilogue. The target location in linear memory must be specified for the R-API, along with the data source. For MMIO, the data is copied into a supplementary buffer, and the dummy registers are modified accordingly. This buffer resides in userspace if Wasm-IO is in userspace. In our prototype, we utilize at least three OS priority levels for $E_{1/4}$ to prevent monopolization by a single Wasm service: the middle priority is designated for enter-calls by a Wasm service. The higher priority is used for Wasm epilogues unless the corresponding Wasm application has already entered $E_{1/4}$. In this case, the lower priority is utilized for the Wasm epilogue. When handling shared interrupts, additional OS priority levels reflect the priorities established during the registration of interrupts via the Wasm service. The Wasm-IO epilogue sets the determined OS thread priority and configures the execution parameters for the Wasm epilogue.

Upon Wasm epilogue execution, i. e., entering $E_{1/4}$, a new execution environment is created for each Wasm module instance. Thus, the context of the current Wasm service is preserved while allowing global data to be shared through the Wasm module instance, e. g., global variables and linear memory. All executing threads are OS threads, the threads proposal [55] is not used.

5 Evaluation

To address Challenge 3 and assess the efficiency of Wasm-IO, we evaluate its temporal isolation and associated overheads. Spatial isolation is assured by Wasm's formal correctness, evaluated in prior works [54, 60]. We aim to demonstrate that Wasm-IO exhibits overheads and WCETs comparable to traditional approaches. We show that upper bounds may be found that enable temporal isolation. Additionally, we evaluate uncertified runtimes leveraging OS protection mechanisms. We illustrate that overheads can be managed with hardware support when using uncertified runtimes.

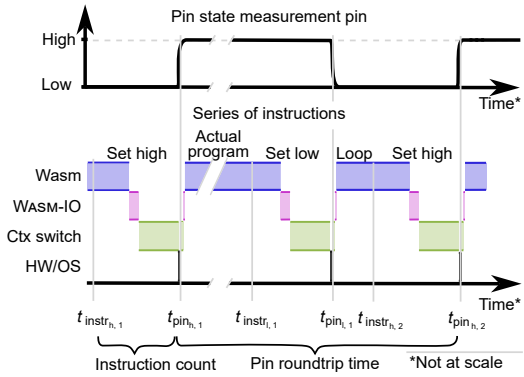


Fig. 8. The measurement process for synchronous peripheral interaction. The I/O instructions are performed on different levels. The external measurement via a GPIO pin only allows cumulative times.

We evaluate by (a) externally measuring the worst-observed execution times and (b) recording the corresponding instruction paths. By combining these recorded paths with the source and

machine code, we derive the necessary (worst-case) path component for WCET analysis (*instruction counts*). When combined with a comprehensive cost model, these instruction counts can effectively be utilized to determine the WCETs [31]. However, since the cost model must incorporate a representation of the runtime, this aspect falls outside the scope of our current work. Instead, our focus is on analyzing loop bounds and, in conjunction with the timing measurements, derive influence on the temporal system behavior and its scaling factors.

We evaluate two scenarios. In the first scenario, we assume that the interpreter is certified, either through verification or comprehensive testing, allowing execution in the kernelspace. We compare the execution of Wasm services to traditional isolation techniques. Specifically, we compare with address space separation using Zephyr's userspace, which employs the Memory Protection Unit (MPU) as a hardware protection mechanism.

In the second scenario, we assume only the OS to be certified. Consequently, the Wasm runtime, providing deployment convenience only, must be isolated utilizing mechanisms provided by the OS. It is evident that executing programs in an environment isolated by both Wasm and userspace leads to suboptimal performance compared to executing solely within userspace. Therefore, we compare execution in userspace, specifically evaluating the impact of DMA acceleration.

We use a STM32WL55JC evaluation board that features a Cortex-M4 CPU running at 48 MHz for conducting our evaluation. The board is equipped with 64 KiB of SRAM, and the peripheral buses also operate at 48 MHz. To avoid probe effects, we use a Saleae Logic Pro 16 logic analyzer, operating at a sampling rate of 250 MHz, for external measurements.

5.1 Synchronous Peripheral Interaction

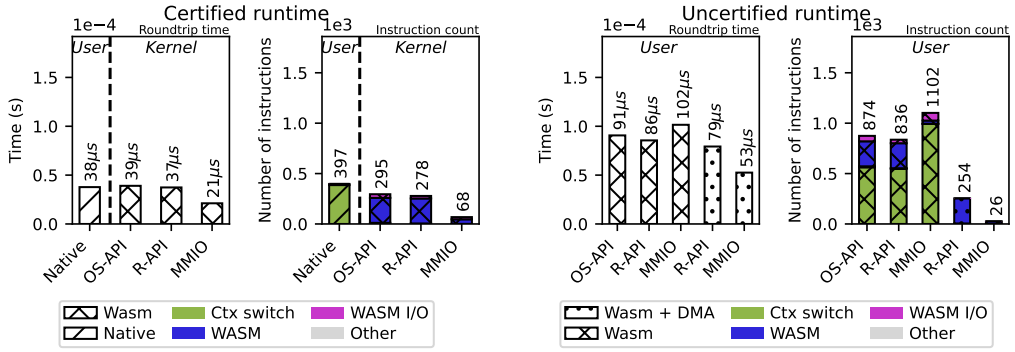
We measure the roundtrip time required to set and reset a GPIO pin (the *measurement pin*) to analyze the costs associated with synchronous peripheral interactions. Figure 8 illustrates the measurement process. The Wasm service executes the software instructions at the Wasm level to change the pin state, e. g., using the R-API call or writing to a memory location. Consequently, the Wasm context is exited. Wasm-IO verifies the memory access and, if successful, transitions into the OS. Following this, the manipulation of the register occurs first at the software level and then at the hardware. Afterward, each invoked function returns involving extensive copying of runtime information across OS (in userspace) and Wasm levels.[3] The same procedure applies when resetting the GPIO pin.

This measurement focuses on the duration between the first I/O instruction in Wasm and the externally observable event, namely, the signal edge. Therefore, the actual program in Figure 8 is omitted (`NOOP`). We only present the WOETs. It is important to note that the measured instruction counts are not directly equivalent to the time measurements, as shown in Figure 8. While they provide a more detailed categorization of instructions, they lack return paths and hardware latency.

The execution of synchronous I/O using the Zephyr API natively in kernelspace yielded a time of 4 µs. However, the execution of code in kernelspace without isolation contradicts the objective of this work and will, therefore, be excluded from further discussions.

Certified Runtime. Figure 9(a) presents the findings of a certified runtime, whereby the Wasm services are executed in kernelspace. We measure the roundtrip time for all three presented approaches: the OS-API, the R-API, and the MMIO, without DMA acceleration. The Zephyr API leveraging userspace isolation and the MPU, serves as a baseline and is denoted as *Native*.

[3]To mitigate Time-Of-Check To Time-Of-Use (TOCTOU) attacks, Zephyr mandates the copying of system call parameters into kernel space. To provide worst-case scenarios, we entirely untrust the userspace runtime, entailing copying of the module instance, execution environment, and memory structures. Undermining these conservative assumptions arguably presents significant opportunities for optimization.



(a) (b)

Fig. 9. (a) The GPIO pin roundtrip time and instruction counts with the Wasm runtime executed in kernelspace and the native userspace execution as baseline. (b) The GPIO pin roundtrip time and instruction counts with the Wasm runtime executed in userspace, with and without DMA.

As illustrated, the OS-API and the R-API are approximately equivalent in duration to the Zephyr API. The MMIO solution is considerably faster than the others, including the native baseline. An analysis of the instruction counts depicted on the right side of Figure 9(a) (compare with Figure 8) indicates that the execution time of the native baseline is predominantly caused by context switching. The OS-API and R-API figures are dominated by instructions related to the Wasm runtime, mainly arising from the transitioning into imported host functions. This process includes the resolution of function targets, the creation of new Wasm call stack frames, and the conversion of function arguments. Conversely, the MMIO approach involves only the standard Wasm bounds check, resulting in markedly reduced execution time and a lower instruction count. Wasm-IO's contribution stems from the device handle lookup or the address check. The slight variations between the OS-API and R-API can be ascribed to the OS-API's additional kernel-level call to the Zephyr API. At the same time, the R-API directly manipulates the register.

The measurement setup entails the discrepancies between timing and instruction counts (OS-API and R-API vs. Zephyr API). As shown in Figure 8, the timing measurements encompass the return paths of the I/O instructions, in contrast to the instruction counts. While these return paths are negligible for the native Zephyr API, they significantly impact the Wasm approaches, where freeing call frames, copying return values, and executing loops within Wasm adds additional instructions.

Our analysis of real-time capabilities emphasizes the substantial dependency on the Wasm runtime. However, we regard this dependency as a component of the cost model. An examination of the recorded traces further indicates that the contribution of Wasm-IO to the worst-case instruction counts is significantly lower than the Wasm runtime. Manual code analysis reveals a linear behavior, scaling by the number of exposed registers across all three methodologies due to searching through them. The current implementation is constrained by a configurable maximum limit.

Uncertified Runtime. In the second scenario, the Wasm runtime is not certified and, therefore, executes in userspace. Figure 9(b) depicts the roundtrip times and instruction counts. As anticipated, the WOETs are considerably worse than in the previous measurement, with the Zephyr API operating in userspace yielding 38 µs. Additionally, the MMIO solution performs slower than all other options due to the extra bookkeeping data that needs to be copied to kernelspace.[4]

[4]The MMIO approach entails the creation of additional temporary kernelspace copies of Wasm bookkeeping data, as memory offsets must be validated in the system call.

The DMA-accelerated approaches show better performance compared to the non-DMA versions, yet they do not surpass the Zephyr API in userspace. The primary factor contributing to overhead is the context switch. The instruction counts for DMA acceleration indicate a reduced CPU burden, as the DMA unit assumes responsibility for copying.

Upper bounds depend on context switch intrinsics, particularly copying fractions of the Wasm structures to the kernelspace, which exhibit a constant size. The contribution of Wasm-IO remains unchanged, compared to the certified case. For the DMA approach, the cost model must account for the external controller, including bus communication.

5.2 Asynchronous Peripheral Interaction—Interrupt Handling

Figure 10 illustrates the measurement process used to determine interrupt latencies. The measurement starts with the rising edge at the interrupt source pin triggering an interrupt. This spawns the Wasm-IO prologue. Following this, the Wasm-IO and Wasm epilogues execute, reading the provided data and activating the measurement pin to signal the data reception. The Zephyr baseline performs the same procedure, utilizing a native callback function to activate the pin after the data is loaded. To calculate the interrupt latency, we take the difference between the trigger edge and the receive edge and subtract the previously measured pin roundtrip time. The gray boxes at the bottom of Figure 10 illustrate the limitations of the instruction count procedure. They indicate the instructions for rescheduling and thread creation which are not included in the instruction count results. Besides, the OS-API does not provide functions for arbitrary data access, which is why it is excluded from the measurement.

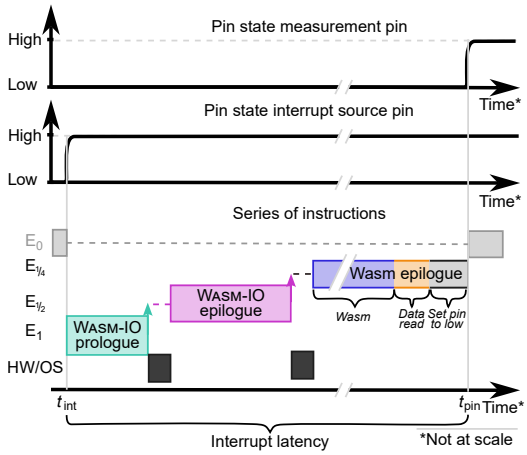


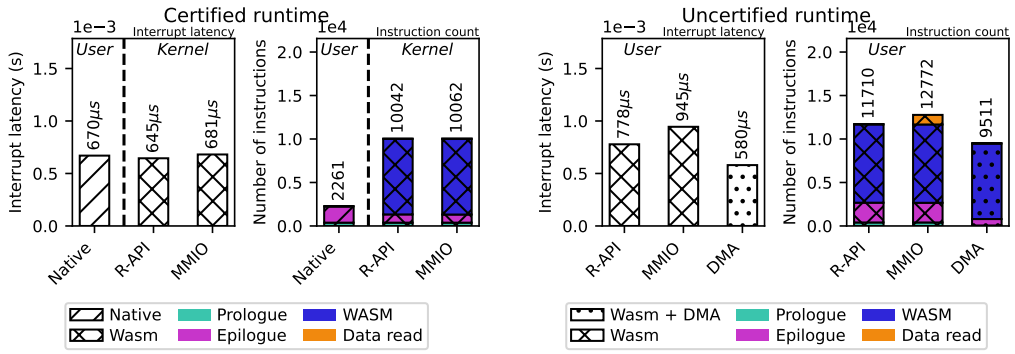
Fig. 10. The measurement for interrupts. The latency encompasses entities on different levels triggered by the interrupt source, captured with a measurement pin.

Certified Runtime. Figure 11(a) presents the figures for the certified case. The left graph shows that all interrupt latencies, including the baseline, are comparable. On the right side, the instruction counts indicate that the duration of the prologue is consistent across all approaches and is relatively brief compared to other components. The unresponsive time spent in the prologue is less than 25 µs, while the epilogue for the Wasm approaches takes less than 63 µs.[5]

In all cases, most instructions are related to the transition into the protected domain, i. e., the userspace or Wasm. The epilogue of the Zephyr baseline is more time-consuming than the Wasm-IO epilogues, as it incorporates the reception callback, which is included in the Wasm epilogues for the other approaches.

The upper latency bounds are again dependent on the OS, its context switch and scheduling implementation, and the invocation of the Wasm runtime. The contribution of Wasm-IO for each Wasm service is directly proportional to the number of registered Wasm epilogues and the number of data items registered for copying. There are configurable maxima that constrain both.

[5]These figures do not account for two kernel thread switches (see Figure 10), as their instruction counts are not measurable in our setup. Including these would further lower the results. We only used our Wasm measurements for calculations, as the absence of context switches invalidates the instruction counts for the Zephyr approach.



(a) (b)

Fig. 11. (a) The interrupt latency measurements and instruction counts with the Wasm runtime executed in kernelspace and the native userspace execution as baseline. (b) The interrupt latency measurements and instruction counts with the Wasm runtime executed in userspace, with and without DMA.

Uncertified Runtime. It is evident that in the case of an uncertified runtime, the observed latencies are higher, as confirmed by Figure 11(b). Despite the fact that the DMA-accelerated approach is faster than the presented ones for the certified runtime, real-time bounds once more hinge on the external hardware module, which requires a detailed cost model. It should be noted that neither the Wasm-IO prologue nor the Wasm-IO epilogue is affected by the runtime trustworthiness.

5.3 Case Study: SPI

To evaluate the practicality of Wasm-IO, we developed an example hardware driver in Wasm that implements an Serial Peripheral Interface (SPI) bus driver. SPI is a widely accepted standard. It has a well-established reference implementation within the OS and available decoders for verification and measurement purposes. However, basic functionality like an SPI driver is typically embedded in the firmware and is *not* implemented as a Wasm service. Our use of this driver is solely for evaluation purposes.

We utilize 1 KiB of data organized into 16-bit words. We configure varying data rates ranging from 18.75 kHz to 24 MHz, equivalent to half of the CPU clock frequency. For measurement, we use the process presented in Figure 8. We set the measurement pin to high before starting the SPI transaction and reset it when it is complete; i. e., the *actual program* of Figure 8 represents the SPI transaction. We subtract the GPIO roundtrip time afterward. Figure 12 presents the results. The corresponding measured data rate is shown on the y-axis for each configured data rate denoted on the x-axis. The values have been normalized with respect to the configured rate. The gray dashed line devoid of markers depicts the execution with the native Zephyr API in kernelspace, indicating the attainable maximum data rate. Even this most performant implementation suffers from a reduction in data

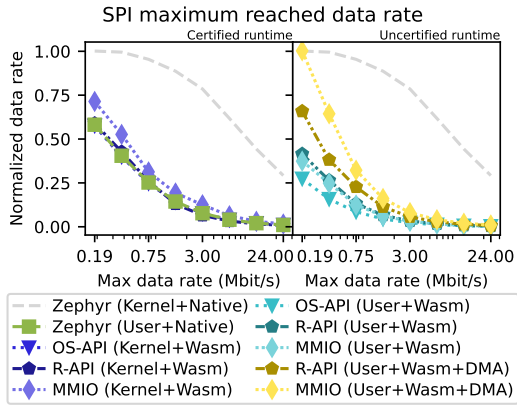


Fig. 12. Execution of an SPI driver in Wasm. Varying data rates are configured, shown on the x-axis, and the reached factor on the y-axis.

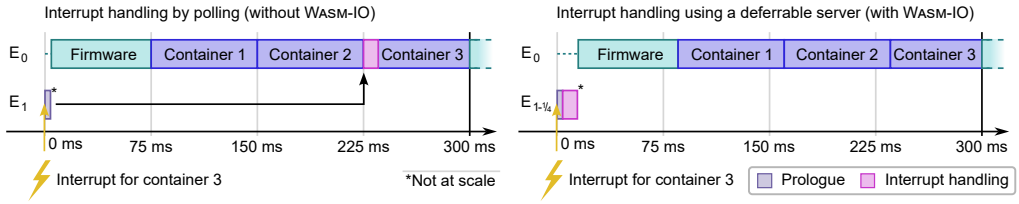


Fig. 13. Exemplary scheduling policy with and without Wasm-IO while maintaining temporal isolation. Through Wasm-IO's extended interrupt model, more timely handling is possible.

rate to 99.5% already at 37.5 kHz, the second lowest data rate. This reduction can be attributed to the relatively low CPU clock frequency compared to the high SPI clock frequency.

The values of the certified runtime are consistent with previous measurements (see Section 5.1). As expected, the MMIO approach performs best, notably even better than the OS-API. Nevertheless, none of these approaches reaches more than 71% of the possible data rate, even if the lowest data rate is configured. Interestingly, in the uncertified case, the OS-API performs worse than both the R-API and MMIO, which contradicts our previous measurement. This results from different implementations of the SPI OS-API and the GPIO OS-API. The former copies references to the Wasm memory, which requires an additional copy of the Wasm linear memory. The latter copies the value of an integer (high or low) to the OS via the stack, which is significantly faster.

The DMA accelerations show remarkably better results as they can copy SPI registers without involving the CPU, thus avoiding transitions from the protection domain for each word. This allows the full data rate, at least for 18.75 kHz, for the MMIO approach. The overall evaluation results show that usage of I/O within Wasm is feasible, but it comes with a considerable, yet bounded overhead. The MMIO approach outperforms the traditional userspace isolation in synchronous I/O and in our SPI case study but requires certified runtimes. Asynchronous I/O shows comparable overheads over all solutions in the certified case. The uncertified case reveals significant performance impacts which impede usage in praxis. DMA acceleration decreases the overall overheads but is accompanied by increasingly complex hardware cost models.

5.4 Practical Considerations Regarding Wasm-IO Overheads

Our evaluation shows that Wasm-IO introduces considerable overheads caused by increased latency, context switching, and runtime interpretation costs. However, it's important to contextualize these figures. Therefore, we first revisit the train control system from the introduction. Data is typically sampled at most every 25 m of track in such systems. A top speed of 300 km/h translates into a sampling period of approximately 300 ms. Every function must be completed within this window, including sensing, processing, and actuation. Figure 13 shows an exemplary system running three containers, each with a separate I/O capability. A common method to manage design-time scheduling in such a multi-container system is round-robin scheduling, resulting in 75 ms time slices for each container. The interrupt handling shown in Figure 13 contrasts two scenarios: polling at the container level and using Wasm-IO's interrupt model. In the polling model, containers are sequentially executed, delaying interrupt handling until the relevant container is scheduled (225 ms in this example). Wasm-IO's interrupt model permits low-latency handling with minimal temporal interference within the system through immediate Wasm prologue and prioritized epilogue execution. This enables the Wasm epilogues to execute independently from the containers' code. It also allows the application of well-established scheduling methods, such as deferred servers or slack stealing. Using server-based scheduling techniques, system designers can precisely specify worst-case overheads (interference), enabling accurate allocation of processing budgets. Furthermore, given that the I/O hardware controls the minimal interarrival time, the spacing between

interrupts is bound (i. e., no double hits), further simplifying worst-case considerations. In the configuration used for evaluation, certified runtimes with MMIO and Wasm-IO experience worst-case interrupt latencies of around 681 µs, closely matching traditional user-level implementations at approximately 671 µs. Uncertified runtimes exceed 945 µs in the same configuration. These latency figures correspond to an additional utilization of about 0.9% for certified and 1.2% for uncertified runtimes per handler within a time slice, indicating that the overhead of Wasm-IO remains within acceptable limits. The cumulative interference scales predictably and remains within deterministic bounds that can be considered in the initial partitioning and allocation of (empty) containers.

In scenarios where certification is unavailable, uncertified runtimes incur significant overheads during synchronous I/O operations. Nevertheless, Wasm-IO enhances modularity, facilitates programming language interoperability, and establishes a well-defined bytecode, which simplifies future-proof updates while maintaining temporal and spatial isolation. This makes Wasm-IO a viable solution for applications with no or minimal synchronous I/O interactions, or for those where I/O is inherently slow due to physical processes. Additionally, application and firmware design may strive to minimize transitions between protection domains. Consequently, achieving a balance between shifting I/O responsibility to the OS for latency optimization and the associated costs of modularity can prove challenging.

In general, the computational overhead of I/O operations is not the predominant aspect of program latencies. The I/O speed of sensors and actuators typically depends more on the duration of the physical processes—such as collecting temperature data or operating motors—rather than communication speed. Moreover, only a limited number of registers (two to four) is utilized after initialization. As they are sorted by access frequency, Wasm-IO checks remain brief. Consequently, experienced overheads are lower than the presented worst-observed execution times.

Furthermore, we used a 48 MHz Cortex-M microcontroller, which, while representative of embedded field devices requiring low-level hardware access, is at the lower end of the performance spectrum. As the latency overhead introduced by Wasm-IO arises from additional instructions, particularly for synchronous I/O, it is proportional to clock cycles. Consequently, latencies for Wasm-IO on higher-performance systems operating in the GHz range with advanced pipelining, caching, and branch prediction are anticipated to be less than one hundredth. Moreover, closer integration of the runtime with the OS may minimize transition durations into kernel space, and fine-grained utilization of hardware features, such as the MPU, could potentially eliminate Wasm-IO's runtime checks, thereby reducing overheads even further.

6 Related Work

To foster the certification process in industrial systems, we introduce Wasm-IO, a framework that ensures spatial and temporal isolation while supporting both synchronous and asynchronous I/O operations. The restrictive, fine-grained, and well-defined nature of Wasm bytecode reduces the trusted computing base, which makes it particularly suitable for certification. Because it is platform-agnostic, developers can verify Wasm-IO against a single standard interface, enabling portability across diverse devices without requiring recertification for each one. By verifying the bytecode and interpreting it directly, we eliminate the need for a certified compiler and still enable optimization. Furthermore, we demonstrate the specific application of well-established concepts-such as Wasm and multi-stage interrupt handling-to ensure safe MMIO and establish temporal isolation using interrupt threads and their priorities. Wasm-IO is focused on I/O isolation, and as such, it shares concepts and design principles with other domains. The following related work tackles similar challenges or applies related foundational principles.

We classify existing work into three categories: *pass-through*, *para-virtualized*, and *fully virtualized I/O*. Pass-through I/O grants the isolated guest direct access to devices, as exemplified by

Wasm-IO's MMIO in kernel space. Para-virtualized I/O splits drivers into host and guest components to enable defined communication, similar to Wasm-IO's OS-API. Fully virtualized I/O systems emulate the behavior of I/O devices. For temporal isolation, Yoo et al. [59] classify devices as dedicated, active, running, and dynamic. Dedicated, active, and running devices inherently deliver interrupts to the active guest without causing interference. In contrast, dynamic devices generate interrupts for inactive guests, which can disrupt the active ones. To prevent information loss, systems must use buffering or rescheduling mechanisms to maintain data integrity. To preserve temporal isolation, systems must sequentialize interrupts and active guests deterministically.

Hypervisors. Hypervisors are a well-established technique for achieving spatial and temporal isolation. They utilize hardware virtualization, allowing multiple guest Virtual Machines (VMs) to operate concurrently. KVM [24] integrates a Virtual Machine Monitor (VMM) into Linux, facilitating hypervisor capabilities. It employs device virtualization for I/O. Any guest VM's I/O access is redirected to userspace, where it interacts with a device model, with KVM operating the underlying hardware. Due to its all-encompassing I/O control, KVM facilitates the implementation of real-time behavior when employing a real-time Linux kernel [49]. For interrupt sequentialization, dedicated threads are allocated for each IRQ, enabling deterministic scheduling, as presented in the Background. For real-time scenarios, Tu et al. [48] forward interrupts directly to designated active VMs. If the VM is inactive, KVM intercepts the interrupt, halts the running VM, and schedules the appropriate one to ensure real-time responsiveness. Quest-V [57] is a separation kernel designed for real-time systems. It leverages hardware-assisted virtualization through Extended Page Tables (EPTs) for spatial isolation and employs Virtual CPUs (vCPUs) for time slicing, providing temporal isolation. Quest-V enforces pass-through I/O by restricting access through EPTs, which provide spatial isolation. In Quest-V, interrupt handling [18] is based on dedicated I/O vCPUs, converting Interrupt Service Routines (ISRs) to threads. Borgioli et al. [15] build on the same concept, using a dedicated I/O VM with pass-through devices as a proxy for other guest VMs. They combine shared memory and ARM QoS-400 hardware regulators, which allow for control over transactions per device per time interval. Thereby, they achieve temporal isolation in communication. In contrast, the Xen [12] hypervisor employs para-virtualization for I/O. It offers simplified device abstractions for guests for implementing drivers, with Xen acting as a proxy. Shared memory and asynchronous buffers provide efficient data transfer. For interrupts, its lightweight event-delivery mechanism stores, aggregates, and processes events on a per-VM basis, with optional callbacks. Ahmad et al. [11] also use shared memory and queues for IRQ buffering with the VMware ESX hypervisor. However, Yoo et al. [59] demonstrate that software buffers undermine temporal isolation. They propose an alternative approach of scheduling the guest VM–rather than the ISR—upon interrupt arrival. They modify Xen accordingly, ensuring timely delivery of interrupts at the cost of other VMs' execution time and the risk of starvation. Cheng and Wang refine this method and present vBalance [16] and hBalance [16], which reduce interrupt latencies in Xen by executing interrupts on running CPUs rather than preempted ones. Pinto et al. [38] developed a hypervisor that leverages ARM's TrustZone. The additional protection layer isolates two VMs: one secure and one non-secure, both with pass-through access to MMIO. Temporal isolation is achieved through hypervisor-level scheduling and pass-through interrupts if the guest is active. However, when the secure VM is active, non-secure interrupts are disabled, leading to missed critical interrupts.

In summary, most existing solutions rely on hardware features often unavailable in constrained embedded systems. They typically require a full OS and target-specific instruction sets, limiting portability and complicating certification. Moreover, para-virtual systems shift isolation responsibilities to the VM creator by requiring modifications to the guest OS. By contrast, Wasm-IO offers lightweight, fine-grained MMIO access control without these dependencies.

Operating System Mechanisms. Operating Systems typically employ multiple protection rings to enforce spatial isolation, with userspace drivers being isolated by definition. Williams et al. [58] propose userspace drivers to minimize the trusted computing base, using a reference validation mechanism-a state machine derived from a device-safe specification-to mediate access to the MMIO area. Similarly, Qiang et al. [39] combine the use of an IOMMU with DMA to enforce isolation. Swift et al. [47] introduce a lightweight kernel-level isolation mechanism that encapsulates kernel functions and validates their parameters. This enables spatial isolation while permitting controlled MMIO access via page table configurations. None of these works addresses temporal behavior. Microkernels refine this paradigm by reducing the kernel to a minimal core and delegating most services, including drivers, to higher protection levels such as userspace. seL4 [23] is a notable example of a formally verified microkernel. Its kernel typically converts interrupts to notifications. However, real-time configurations disable asynchronous interrupt delivery to userspace, necessitating polling. The authors demonstrate temporal isolation and real-time applicability via static analysis [13]. However, these approaches are inherently tied to specific OSes and instruction sets, which constrains platform independence and increases certification complexity. Moreover, their isolation mechanisms operate at a coarser granularity than those of Wasm-IO.

Software-based Isolation. Moreover, software-level isolation strategies can be implemented through programming languages, intermediate representations, or runtime systems, often supported by hardware acceleration. Rust is a prominent example of a type-safe language leveraging compiler mechanisms. Embedded Rust [10] encapsulates MMIO in structured types, enabling the compiler to enforce type, ownership, and lifetime constraints. Several projects extend this model with support for scheduling and interrupts. For instance, Embassy [1] integrates async-await execution with interrupt handling through coroutines. However, the cooperative scheduling nature of coroutines makes this incompatible with Wasm-IO. Tock [26], an OS for low-power systems, transforms interrupts into events processed by a cooperative scheduler, establishing a multi-stage handling approach. RTFM [19], now RTIC [3], advances this design using hardware-supported multi-level interrupt handling for scheduling and event management. However, compiling to native code shifts verification responsibility to the operator and complicates safety assurance, making it incompatible with Wasm-IO's objectives.

Managed languages like Java offer platform independence and spatial isolation through their runtime environments. The Java Virtual Machine (JVM)'s type system and runtime checks enforce memory safety. The Real-Time Specification of Java (RTSJ) [14] extends Java for real-time use by introducing the `RawMemoryAccess` class for MMIO and `AsyncEvents` for interrupt handling, modeled as sequential threads. Schoeberl et al. [41] provide RTSJ-compliant packages with `HardwareObjects` using volatile fields to represent registers. Depending on the implementation, bytecode instructions such as `get/putfield` can reference MMIO base addresses directly. Interrupts are managed via a two-level model: a low-level handler triggers a higher-level event handler. KESO [46] offers similar capabilities for constrained real-time devices, based on AOT compilation. While Java aligns with Wasm-IO in terms of isolation and portability, the tight coupling between the runtime and language limits compatibility with other ecosystems. Its flexible bytecode—e. g., opcodes like `invokedynamic` and `goto`—introduces control-flow variance that complicates static verification, unlike the more rigid structure of Wasm. The eBPF bytecode also supports isolation. Femto-Containers [61] present an eBPF-based runtime for low-power devices. They rely on the OS for I/O but do not address temporal isolation. eBPF restricts programming expressiveness, prohibiting features such as floating-point operations and recursion. Languages like MicroPython [2] and Lua [35] prioritize rapid prototyping and offer broad I/O support. However, they lack built-in isolation mechanisms. The integration of Wasm into embedded systems has driven research on I/O and runtime models. AOT compilation is often preferred due to its low runtime overhead

and memory footprint. eWASM [37] targets embedded systems with an AOT compiler and built-in bounds checking. It uses FreeRTOS for temporal isolation, though it does not focus on fine-grained I/O control. Li et al. [27, 29, 30] propose custom APIs for basic I/O in Wasm, replacing API calls with native host functions to avoid sandbox transitions. However, these approaches do not address temporal isolation. Aerogel [32] introduces abstract `sensing` and `acting` interfaces in place of MMIO. It performs runtime access checks based on metrics like energy usage and duration. Unlike Wasm-IO, Aerogel uses fixed I/O addresses specified in an external configuration file.

As explained, AOT compilation complicates the certification process—also echoed in safety-critical domains using interpreters. Studies typically expose the host API for I/O operations. Zaeske et al. [60] employ Wasm and the wasmi interpreter in Avionics for software reuse and expose the ARINC 653 API. They achieve temporal isolation via the underlying hypervisor and modify the interpreter to consume "fuel" during operation. Silverline [40] is an orchestration framework that leverages the WAMR interpreter for small, real-time devices handling I/O through the host system's ctrlX or Vehicle Signal Specification interface. Temporal isolation is provided by the OS. Finally, WARDuino [44] enhance programmability by integrating Wasm into Arduino devices with the wac interpreter and exposing the Arduino API. Notably, Wasm-IO is the first work that addresses interrupt handling and MMIO in the context of Wasm, as exposing the OS's API is inadequate for extension of industrial automation systems, particularly regarding device drivers. WASI [20] aims to standardize a uniform system interface. However, as it remains in its preview phase and undergoes significant modifications, concerns regarding its long-term stability persist, alongside with issues related to a consistent Application Binary Interface (ABI) and potential overheads [43, 60].

7 Conclusion

In this article, we presented how Wasm-IO enables software extensions of constrained devices, including low-level device drivers. Our approach facilitates the integration of cross-platform hardware requirements by labeling peripheral dependencies. Our prototype derives these labels from devicetree nodes. At load time, the corresponding MMIO regions are rendered available to Wasm services, contingent upon marking accessability at firmware design time by our novel devicetree binding. Wasm-IO enables synchronous I/O, implementing multiple approaches, each ensuring access is restricted to these explicitly provided regions.

Moreover, we introduced interrupt handling into Wasm. We extended the traditional system model by another priority level $E_{1/4}$, dedicated explicitly to Wasm. With this separation, we introduce (a) a constant overhead per interrupt handler per Wasm service on E_0 and $E_{1/2}$ and (b) limits on the number of handlers and Wasm services to constrain the overall execution time during firmware design. Thereby, Wasm-IO provides means to decouple the temporal influence associated with a Wasm service from the remaining system, enabling more timely interrupt handling for Wasm.

In our evaluation, we presented how hardware interaction and interrupt latencies and the costs associated with containerization can be pushed to practical levels while preserving strong isolation. The results demonstrate that the MMIO-based approach is advantageous when writing to registers. Hardware acceleration, e. g., by the proposed DMA approach, helps significantly increase data rates but has the disadvantage of a more complex hardware cost model. In our case study, we show practical applicability and—revisiting the initial example—demonstrate that overheads remain within acceptable limits. It can be concluded that hardware configuration and instrumentation can be performed from a Wasm service in a bounded time. However, hardware acceleration should be taken into account for performance reasons when accordable with real-time demands. While formally verified runtimes enhance proof of correctness, reliance on them may be overly stringent. Until future research provides formally correct runtimes for resource-constrained devices, we recommend certifying runtimes by demonstrating standard conformance through rigorous testing.

With Wasm-IO, we effectively tackle the initial challenges of containerized systems in the automation industry, enabling the integration of drivers for novel peripherals into existing systems without the need of recertification. Thereby, Wasm-IO extends the lifespan of long-life industrial devices. Moreover, Wasm-IO seamlessly brings the conveniences of containerization—with benefits widely recognized in cloud development—one step closer to industrial shop floors.

References

[1] 2025. Embassy Book. Retrieved from https://embassy.dev/book/. Last accessed: 2025-06-12.

[2] 2025. MicroPython. Retrieved from http://micropython.org/. Last accessed: 2025-06-13.

[3] 2024. Real-Time Interrupt-driven Concurrency (RTIC). Retrieved from https://github.com/rtic-rs/rtic. Last accessed: 2025-06-12.

[4] 2010. IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems. International Electrical Commission (IEC).

[5] 2020. EN 50128: Railway Applications - Communications, Signalling and Processing Systems - Software for Railway Control and Protection Systems. Comité Européen de Normalisation Électrotechnique (CENELEC).

[6] 2019. WebAssembly Specification 1.0 Retrieved from https://www.w3.org/TR/wasm-core-1/. Last accessed: 2023-09-07.

[7] 2023. Devicetree Specification. Retrieved March 3, 2023 from https://github.com/devicetree-org/devicetree-specification/releases/v0.4

[8] 2023. Understand the WAMR Stacks. Retrieved October 2, 2024 from https://bytecodealliance.github.io/wamr.dev/blog/understand-the-wamr-stacks/

[9] 2023. The WAMR Memory Model. Retrieved October 2, 2024 from https://bytecodealliance.github.io/wamr.dev/blog/the-wamr-memory-model/

[10] 2025. The Embedded Rust Book. Retrieved June 12, 2025 from https://docs.rust-embedded.org/book/

[11] Irfan Ahmad, Ajay Gulati, and Ali Mashtizadeh. 2011. vIC: Interrupt coalescing for virtual machine storage device IO. In *Proceedings of the 2011 USENIX Annual Technical Conference*. USENIX Association, Portland, USA, 45–58.

[12] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. 2003. Xen and the art of virtualization. *ACM SIGOPS Operating Systems Review* 37, 5 (Oct. 2003), 164–177. DOI: https://doi.org/10.1145/1165389.945462

[13] Bernard Blackham, Yao Shi, Sudipta Chattopadhyay, Abhik Roychoudhury, and Gernot Heiser. 2011. Timing analysis of a protected operating system kernel. In *Proceedings of the 2011 IEEE 32nd Real-Time Systems Symposium* 339–348. DOI: https://doi.org/10.1109/RTSS.2011.38

[14] Greg Bollella, Ben Brosgol, Peter Dibble, Steve Furr, James Gosling, David Hardin, and Mark Turnbull. 2000. *The Real-Time Specification for Java*. Addison-Wesley. Retrieved from https://jcp.org/aboutJava/communityprocess/first/jsr001/rtj.pdf

[15] Niccolò Borgioli, Matteo Zini, Daniel Casini, Giorgiomaria Cicero, Alessandro Biondi, and Giorgio Buttazzo. 2022. An I/O virtualization framework with I/O-related memory contention control for real-time systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (Nov. 2022), 4469–4480. DOI: https://doi.org/10.1109/TCAD.2022.3202434

[16] Luwei Cheng and Cho-Li Wang. 2012. vBalance: Using interrupt load balance to improve I/O performance for SMP virtual machines. In *Proceedings of the 3rd ACM Symposium on Cloud Computing (SoCC'12)*. ACM, New York, NY, USA, 1–14. DOI: https://doi.org/10.1145/2391229.2391231

[17] Sam Clegg, W3C WebAssembly Community Group, and W3C WebAssembly Working Group. 2024. WebAssembly Proposal: Memory64. Retrieved March 7, 2025 from https://github.com/WebAssembly/memory64

[18] Matthew Danish, Ye Li, and Richard West. 2011. Virtual-CPU scheduling in the quest operating system. In *Proceedings of the 2011 17th IEEE Real-Time and Embedded Technology and Applications Symposium*. 169–179. DOI: https://doi.org/10.1109/RTAS.2011.24

[19] Johan Eriksson, Fredrik Häggström, Simon Aittamaa, Andrey Kruglyak, and Per Lindgren. 2013. Real-time for the masses, step 1: Programming API and static priority SRP kernel primitives. In *Proceedings of the 2013 8th IEEE International Symposium on Industrial Embedded Systems (SIES)*. 110–113. DOI: https://doi.org/10.1109/SIES.2013.6601482

[20] Dan Gohman, Lin Clark, Alex Crichton, Andrew Brown, Sam Clegg, Pat Hickey, Yosh, Dave Bakker, Colin Ihrig, YAMAMOTO Yuji, et al. 2024. WebAssembly/WASI: V0.2.2. DOI: https://doi.org/10.5281/zenodo.13888155. Accessed Date: March 5, 2025.

[21] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. 2017. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. ACM, New York, NY, USA, 185–200. DOI: https://doi.org/10.1145/3062341.3062363

[22] Heejin Ahn, J. F. Bastien, Dan Gohman, and Derek L. Schuff. 2028. WebAssembly/Tool-Conventions: C++ Volatile Support. Retrieved March 5, 2025 from https://github.com/WebAssembly/tool-conventions/issues/60

[23] Gernot Heiser. 2025. The seL4 Microkernel – An Introduction. The seL4 Foundation.

[24] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. 2007. Kvm: The Linux virtual machine monitor. In Proceedings of the Linux Symposium, Vol. 1.

[25] Ryan Kurte. 2023. Embedded WASM. Retrieved March 7, 2025 from https://github.com/embedded-wasm

[26] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B. Giffin, Pat Pannuto, Prabal Dutta, and Philip Levis. 2017. Multiprogramming a 64kB computer safely and efficiently. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP'17)*. ACM, New York, NY, USA, 234–251. DOI: https://doi.org/10.1145/3132747.3132786

[27] Borui Li, Wei Dong, and Yi Gao. 2021. WiProg: A WebAssembly-based approach to integrated IoT programming. In *IEEE INFOCOM 2021 - Proceedings of the IEEE Conference on Computer Communications*. 1–10. DOI: https://doi.org/10.1109/INFOCOM42981.2021.9488424

[28] Borui Li, Hongchang Fan, Yi Gao, and Wei Dong. 2021. ThingSpire OS: A WebAssembly-based IoT operating system for cloud-edge integration. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*. DOI: https://doi.org/10.1145/3458864.3466910

[29] Borui Li, Hongchang Fan, Yi Gao, and Wei Dong. 2022. Bringing webassembly to resource-constrained IoT devices for seamless device-cloud integration. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*. DOI: https://doi.org/10.1145/3498361.3538922

[30] Borui Li, Hongchang Fan, Yi Gao, and Wei Dong. 2024. WAWOT: Towards flexible and efficient web of things services via webassembly on resource-constrained IoT devices. *IEEE Transactions on Computers* 74, 3 (2024), 1–14. DOI: https://doi.org/10.1109/TC.2024.3500385

[31] Yau-Tsun Steven Li and Sharad Malik. 1995. Performance analysis of embedded software using implicit path enumeration. *SIGPLAN Notices* 30, 11 (Nov. 1995), 88–98. DOI: https://doi.org/10.1145/216633.216666

[32] Renju Liu, Luis Garcia, and Mani Srivastava. 2021. Aerogel: Lightweight access control framework for webassembly-based bare-metal IoT devices. In *Proceedings of the 2021 IEEE/ACM Symposium on Edge Computing (SEC)*. 94–105. DOI: https://doi.org/10.1145/3453142.3491282

[33] Robert Love. 2010. *Linux Kernel Development* (3rd ed.). Addison-Wesley.

[34] Niko Mäkitalo, Tommi Mikkonen, Cesare Pautasso, Victor Bankowski, Paulius Daubaris, Risto Mikkola, and Oleg Beletski. 2021. WebAssembly modules as lightweight containers for liquid IoT applications. In *Web Engineering*, Marco Brambilla, Richard Chbeir, Flavius Frasincar, and Ioana Manolescu (Eds.). Springer Intl. Publishing, Cham, 328–336. DOI: https://doi.org/10.1007/978-3-030-74296-6_25

[35] Bogdan Marinescu, Dado Sutter, and James Snyder. 2013. eLua. Retrieved from https://eluaproject.net/. Last accessed: 2025-06-13.

[36] Jämes Ménétrey, Marcelo Pasin, Pascal Felber, and Valerio Schiavoni. 2021. Twine: An embedded trusted runtime for WebAssembly. In *Proceedings of the 2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 205–216. DOI: https://doi.org/10.1109/ICDE51399.2021.00025

[37] Gregor Peach, Runyu Pan, Zhuoyi Wu, Gabriel Parmer, Christopher Haster, and Ludmila Cherkasova. 2020. eWASM: Practical software fault isolation for reliable embedded devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (Nov. 2020), 3492–3505. DOI: https://doi.org/10.1109/TCAD.2020.3012647

[38] Sanndro Pinto, Hugo Araujo, Daniel Oliveira, José Martins, and Adriano Tavares. 2019. Virtualization on trustzone-enabled microcontrollers? Voilà!. In *Proceedings of the 2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 293–304. DOI: https://doi.org/10.1109/RTAS.2019.00032

[39] Weizhong Qiang, Kang Zhang, and Hai Jin. 2016. Reducing TCB of linux kernel using user-space device driver. In *Algorithms and Architectures for Parallel Processing*. Jesus Carretero, Javier Garcia-Blas, Ryan K.L. Ko, Peter Mueller, and Koji Nakano (Eds.), Springer Intl. Publishing, Cham, 572–585. DOI: https://doi.org/10.1007/978-3-319-49583-5_45

[40] Arjun Ramesh, Tianshu Huang, Emily Ruppel, Dakshina Dasari, Behnaz Pourmohseni, Fedor Smirnov, Marco Giani, Paolo Pazzaglia, Charles Shelton, Nuno Pereira, et al. 2025. Silverline: Lightweight virtualization and orchestration of distributed systems. In *Proceedings of the 2025 IEEE 31st Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 375–388. DOI: https://doi.org/10.1109/RTAS65571.2025.00042

[41] Martin Schoeberl, Stephan Korsholm, Tomas Kalibera, and Anders P. Ravn. 2011. A hardware abstraction layer in Java. *ACM Transactions on Embedded Computing Systems* 10, 4 (Nov. 2011), 42:1–42:40. DOI: https://doi.org/10.1145/2043662.2043666

[42] Wolfgang Schröder-Preikschat. 1994. *The Logical Design of Parallel Operating Systems*. Prentice Hall Intl., Upper Saddle River, NJ, USA. DOI: https://doi.org/10.5555/184212

[43] SIG Embedded. 2025. WASI / Wasm System Structure Exploration of Improvements or Alternative Solutions. Retrieved February 24, 2025 from https://github.com/woodsmc/sig-embedded/blob/8e88d8241dc1a222d1ff39da08fde46c43a0e722/ExploringOptions/Alternative_Solutions_(WASI_Wasm).md

[44] Robbert Gurdeep Singh and Christophe Scholliers. 2019. WARDuino: A dynamic WebAssembly virtual machine for programming microcontrollers. In *Proceedings of the 16th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes*. DOI: https://doi.org/10.1145/3357390.3361029

[45] David John Smith and Kenneth G. L. Simpson. 2011. *Safety Critical Systems Handbook. A Straightforward Guide to Functional Safety, IEC 61508 (2010 Edition) and Related Standards ; Including: Process IEC 61511, Machinery IEC 62061 and ISO 13849* (3rd ed.). Elsevier, Butterworth-Heinemann, Amsterdam [u.a.].

[46] Michael Stilkerich, Isabella Thomm, Christian Wawersich, and Wolfgang Schröder-Preikschat. 2012. Tailor-made JVMs for statically configured embedded systems. *Concurrency and Computation: Practice and Experience* 24, 8 (2012), 789–812. DOI: https://doi.org/10.1002/cpe.1755

[47] Michael M. Swift, Brian N. Bershad, and Henry M. Levy. 2003. Improving the reliability of commodity operating systems. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP'03)*. ACM, New York, NY, USA, 207–222. DOI: https://doi.org/10.1145/945445.945466

[48] Cheng-Chun Tu, Michael Ferdman, Chao-tung Lee, and Tzi-cker Chiueh. 2015. A comprehensive implementation and evaluation of direct interrupt delivery. *SIGPLAN Notices* 50, 7 (Mar. 2015), 1–15. DOI: https://doi.org/10.1145/2817817.2731189

[49] Rik van Riel. 2015. Real-Time KVM from the Ground Up. Retrieved June 11, 2025 from https://wiki.linuxfoundation.org/_media/realtime/events/rt-summit2016/kvm_rik-van-riel.pdf

[50] Stefan Wallentowitz, Bastian Kersting, and Dan Mihai Dumitriu. 2022. Potential of WebAssembly for embedded systems. In *Proceedings of the 2022 11th Mediterranean Conference on Embedded Computing (MECO)*. 1–4. DOI: https://doi.org/10.1109/MECO55406.2022.9797106

[51] Wenwen Wang. 2022. How far we've come – A characterization study of standalone WebAssembly runtimes. In *Proceedings of the 2022 IEEE International Symposium on Workload Characterization (IISWC)*. 228–241. DOI: https://doi.org/10.1109/IISWC55918.2022.00028

[52] Conrad Watt. 2018. Mechanising and verifying the WebAssembly specification. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2018)*. ACM, New York, NY, USA, 53–65. DOI: https://doi.org/10.1145/3167082

[53] Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, and Philippa Gardner. 2021. Two mechanisations of WebAssembly 1.0. In *Formal Methods*. Marieke Huisman, Corina Păsăreanu, and Naijun Zhan (Eds.), Springer Intl. Publishing, Cham, 61–79. DOI: https://doi.org/10.1007/978-3-030-90870-6_4

[54] Conrad Watt, Maja Trela, Peter Lammich, and Florian Märkl. 2023. WasmRef-isabelle: A verified monadic interpreter and industrial fuzzing oracle for WebAssembly. *Proceedings of the ACM on Programming Languages* 7, PLDI (June 2023), 110:100–110:123. DOI: https://doi.org/10.1145/3591224

[55] Conrad Watt, W3C WebAssembly Community Group, and W3C WebAssembly Working Group. 2024. WebAssembly Proposal: Threads. Retrieved March 7, 2025 from https://github.com/WebAssembly/threads

[56] Elliott Wen and Gerald Weber. 2020. Wasmachine: Bring IoT up to speed with a WebAssembly OS. In *Proceedings of the 2020 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. 1–4. DOI: https://doi.org/10.1109/PerComWork.s48775.2020.9156135

[57] Richard West, Ye Li, Eric Missimer, and Matthew Danish. 2016. A virtualized separation kernel for mixed-criticality systems. *ACM Transactions on Computer Systems* 34, 3 (June 2016), 8:1–8:41. DOI: https://doi.org/10.1145/2935748

[58] Dan Williams, Patrick Reynolds, Kevin Walsh, Emin Gun Sirer, and Fred B. Schneider. 2008. Device driver safety through a reference validation mechanism. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation*. .

[59] Seehwan Yoo, Miri Park, and Chuck Yoo. 2009. A step to support real-time in virtual machine. In *Proceedings of the 2009 6th IEEE Consumer Communications and Networking Conference*. 1–7. DOI: https://doi.org/10.1109/CCNC.2009.4784876

[60] Wanja Zaeske, Sven Friedrich, Tim Schubert, and Umut Durak. 2023. WebAssembly in avionics : Decoupling software from hardware. In *Proceedings of the 2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*. 1–10. DOI: https://doi.org/10.1109/DASC58513.2023.10311207

[61] Koen Zandberg, Emmanuel Baccelli, Shenghao Yuan, Frédéric Besson, and Jean-Pierre Talpin. 2022. Femto-containers: Lightweight virtualization and fault isolation for small software functions on low-power IoT microcontrollers. In *Proceedings of the 23rd ACM/IFIP International Middleware Conference (Middleware'22)*. ACM, New York, NY, USA, 161–173. DOI: https://doi.org/10.1145/3528535.3565242

Received 7 August 2025; revised 7 August 2025; accepted 7 August 2025